

SAE Intégrative

Documentation Technique

Table des matières

■ views.py	Calcul des moyennes, dashboard, filtrage, export CSV, CRUD
■ ■ models.py	Modèles Capteurs et Donnees, relations, clés étrangères
■ ■ JavaScript	Rafraîchissement automatique de la page et du graphique

1. views.py

■ 1.1 – Calcul des moyennes

En Django, ajouter `.values()` juste après un `.objects` modifie le comportement de la requête : au lieu de récupérer des objets complets, Django regroupe les lignes qui partagent la même valeur dans le champ spécifié (`id_capteur__emplacement`), réduisant ainsi la consommation de RAM.

Une fois les groupes formés par pièce, `.annotate()` applique une fonction mathématique sur chaque groupe. On utilise `Avg('temperature')` pour calculer la moyenne de chaque pièce. La requête SQL équivalente :

```
SELECT id_capteur__emplacement, AVG(temperature)
FROM donnees
GROUP BY id_capteur__emplacement;
```

■ 1.2 – Affichage des 5 dernières données (dashboard)

Par défaut, l'accès aux clés étrangères dans une boucle génère le problème des **requêtes N+1**. Pour l'éviter, on utilise `select_related` qui force une requête SQL avec `JOIN`, passant ainsi de 6 requêtes à **une seule**, peu importe le volume de données affiché.

■ 1.3 – Filtrage

Django effectue trois actions en une seule ligne :

- `filter(id_capteur__emplacement=salle_nom)` : La double underscore (__) traverse les tables : de Donnees → Capteurs → colonne emplacement.
- `.select_related('id_capteur')` : Jointure SQL : embarque immédiatement les infos du capteur pour éviter le N+1.
- `.order_by('-timestamp')` : Le - indique un tri décroissant : mesures les plus récentes en premier.

Recherche dynamique : grâce aux objets `Q()` et au connecteur `| (OU)`, on cherche dans l'identifiant brut ou dans le nom du capteur. `__icontains` est insensible à la casse (équivalent `LIKE %mot%` en SQL).

■ 1.4 – Rafraîchissement automatique de la page (JavaScript)

```
setInterval(() => window.location.reload(), 30000);
```

`setInterval(action, délai)` répète une action toutes les X ms. Ici, `window.location.reload()` recharge la page toutes les **30 secondes** (30 000 ms).

■ 1.5 – Rafraîchissement du graphique (fenêtre glissante 2h)

```
if not date_debut and not date_fin:
    il_y_a_2_heures = timezone.now() - timedelta(hours=2)
    mesures_chrono = mesures_queryset.filter(
        timestamp__gte=il_y_a_2_heures
    ).order_by('timestamp')
else:
    mesures_chrono = mesures_queryset.order_by('timestamp')
```

Sans filtre : fenêtre glissante limitée aux 2 dernières heures via **timedelta(hours=2)** soustrait à **timezone.now()**. **Avec filtre** : la restriction est levée, toutes les données demandées sont transmises. Dans les deux cas, **.order_by('timestamp')** trie les points de gauche à droite pour Chart.js.

■ 1.6 – Export CSV

Le bouton d'export réinjecte les filtres actifs dans l'URL (search, date_debut, date_fin), garantissant que le fichier CSV correspond exactement à la sélection en cours.

```
def export_salle_csv(request, salle_nom):
    search_query = request.GET.get('search', '').strip()
    date_debut = request.GET.get('date_debut', '')
    date_fin = request.GET.get('date_fin', '')

    mesures_queryset = (
        Donnees.objects
        .filter(id_capteur__emplacement=salle_nom)
        .select_related('id_capteur')
        .order_by('-timestamp')
    )

    if search_query:
        mesures_queryset = mesures_queryset.filter(
            Q(id_capteur__id_capteur__icontains=search_query) |
            Q(id_capteur__nom__icontains=search_query)
        )

    if date_debut:
        mesures_queryset = mesures_queryset.filter(timestamp__gte=date_debut)
    if date_fin:
        mesures_queryset = mesures_queryset.filter(timestamp__lte=date_fin)

    # Réponse HTTP forcée en téléchargement CSV
    response = HttpResponse(content_type='text/csv')
    response['Content-Disposition'] = (
        f'attachment; filename="export_{salle_nom}.csv"'
    )
    writer = csv.writer(response, delimiter=';') # ; = compat Excel FR
    writer.writerow(['ID Capteur', 'Nom', 'Temperature (C)', 'Date'])
```

■ 1.7 – Fonctions CRUD (Supprimer / Modifier)

Suppression : `get_object_or_404` valide l'ID avant de supprimer, renvoyant une 404 propre si invalide. Après suppression, redirection automatique vers le tableau de bord de la pièce concernée.

```
def supprimer_donnee(request, donnee_id):
    donnee = get_object_or_404(Donnees, id_donnee=donnee_id)
    salle_nom = donnee.id_capteur.emplacement
    donnee.delete()
    return redirect('salle_detail', salle_nom=salle_nom)
```

Modification : GET pré-remplit le formulaire, POST écrase les valeurs et appelle `.save()` → requête SQL **UPDATE** en base.

```
def modifier_capteur(request, capteur_id):
    capteur = get_object_or_404(Capteurs, id_capteur=capteur_id)

    if request.method == 'POST':
        capteur.nom = request.POST.get('nom_capteur')
        capteur.emplacement = request.POST.get('emplacement_capteur')
        capteur.save() # → SQL UPDATE
        return redirect('salle_detail', salle_nom=capteur.emplacement)

    return render(request, 'capteurs/modifier_capteur.html',
                  {'capteur': capteur})
```

2. models.py

■ 2.1 – Modèle Capteurs

```
class Capteurs(models.Model):
    id_capteur = models.CharField(primary_key=True, max_length=50)
    nom = models.CharField(unique=True, max_length=100,
                           blank=True, null=True)
    emplacement = models.CharField(max_length=100,
                                   blank=True, null=True)

    class Meta:
        managed = False # table gérée manuellement
        db_table = 'capteurs'
```

id_capteur (CharField, PK) : clé primaire textuelle calquée sur l'identifiant physique ou l'adresse MAC du capteur IoT.

emplacement : nom de la pièce (ex : « chambre1 »). C'est sur ce champ que repose toute la logique de filtrage des vues.

■ 2.2 – Modèle Donnees

```
class Donnees(models.Model):
    id_donnee = models.AutoField(primary_key=True)
    timestamp = models.DateTimeField() # horodatage de la trame
    temperature = models.FloatField() # mesure brute
    id_capteur = models.ForeignKey(
        Capteurs,
        on_delete=models.DO_NOTHING, # PAS de cascade !
        db_column='id_capteur',
        blank=True, null=True
    )

    class Meta:
        managed = False
        db_table = 'donnees'
```

timestamp : indispensable pour le tracé chronologique Chart.js (tri ASC).

temperature (FloatField) : mesure physique brute en nombre à virgule.

■ 2.3 – Pourquoi DO_NOTHING ?

Avec **CASCADE**, supprimer un capteur détruirait instantanément des millions de lignes d'historique thermique. **DO_NOTHING** garantit que les archives de mesures sont conservées quoi qu'il arrive, même si la fiche du capteur est temporairement retirée.

Récapitulatif des champs clés

Champ	Type	Rôle
id_capteur	CharField (PK)	Identifiant physique du capteur IoT
emplacement	CharField	Nom de la pièce — base du filtrage
id_donnee	AutoField (PK)	Clé primaire auto-incrémentée
timestamp	DateTimeField	Horodatage de la mesure
temperature	FloatField	Valeur brute en virgule flottante
FK id_capteur	ForeignKey (DO_NOTHING)	Lien Donnees → Capteurs, sans cascade